

**AI-POWERED UNIFIED MOBILITY AND DELIVERY SYSTEM
USING A DYNAMIC MULTI-SERVICE FEASIBILITY ENGINE**

A PROJECT REPORT

Submitted by

SRINITHYA M	710722104102
SRIRAM V	710722104103
VEERA SIVA SAKTHI A M	710722104116

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Dr. N.G.P INSTITUTE OF TECHNOLOGY, COIMBATORE - 641048

AN AUTONOMOUS INSTITUTION

ANNA UNIVERSITY: CHENNAI 600 025

MARCH 2026

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this Report titled “**AI-POWERED UNIFIED MOBILITY AND DELIVERY SYSTEM USING A DYNAMIC MULTI-SERVICE FEASIBILITY ENGINE**” is the bonafide work of **SRINITHYA M (710722104102)**, **SRIRAM V (710722104103)** and **VEERA SIVA SAKTHI A M (710722104116)** who carried out the work under my supervision.

SIGNATURE

Dr. J. POONGODI M.E. (Ph. D)
SUPERVISOR
Assistant Professor,
Department of Computer Science and
Engineering,
Dr. N. G. P Institute of Technology,
Coimbatore-641048.

SIGNATURE

Dr. V. PRAVEENA M.E, Ph.D.
HEAD OF THE DEPARTMENT
Professor,
Department of Computer Science and
Engineering,
Dr. N. G. P Institute of Technology,
Coimbatore-641048.

Submitted for the Final Year Project Work Viva Voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

Words act as a gateway to express tokens of acknowledgement. First of all, we would like to thank the supreme power, our **Parents** and **Almighty God**, who has given us the strength and courage to complete our work successfully.

We would like to express our profound gratitude and deep sense of thanks to **Dr. Nalla G. Palaniswami MD., AB (USA)**, Chairman of Kovai Medical Center and Hospital, for providing us with the necessary facilities to complete our project work effectively.

Our heartfelt gratitude to **Dr. Thavamani D Palaniswami MD., AB (USA)**, Secretary of Dr. N. G. P. Institute of Technology, for her generous attitude and motivation, which had been one of the reasons to complete our project.

We are sincerely grateful to **Dr. S. U. Prabha M.E, Ph.D.** Principal who has always been a source of inspiration, well-wisher and a pillar of support for all the students in our institution by rendering full motivation always.

We are sincerely thankful to **Dr. D. Palanikkumar M.E, Ph.D.** Professor and Dean of Computing Science, for his continuous support, valuable guidance, and encouragement during our project work.

We are highly indebted to **Dr. V. Praveena M.E, Ph.D.** Professor and Head of Computer Science and Engineering Department, for her dedication, interest and overwhelming attitude to help students that have helped us to a great extent to accomplish this task.

We wish to thank our Project Coordinator **Dr. N. Krishnammal M.E, Ph.D.** Assistant Professor, Department of Computer Science and Engineering, for her excellent assistance, guidance, regular feedback and constructive ideas.

We express our heartfelt thanks to our Project Guide **Dr. J. Poongodi M.E, Ph.D.** Assistant professor, Department of Computer Science and Engineering, for her valuable guidance and timely help for completing our project.

Finally, we owe huge thanks to our teaching **Faculty members** and **Non-teaching** staff members, whose love and insights have enriched our work.

ABSTRACT

Urban transport platforms today operate as isolated networks. Ride-hailing, food delivery and parcel courier services each run their own fleet, their own matching logic and their own routing pipeline, with no visibility into one another. The result is a large volume of empty running: a driver who has just dropped a passenger returns without a payload, while a parcel waiting a few hundred metres away is assigned to a second vehicle. This project proposes the design and development of an AI-Powered Unified Mobility and Delivery System that removes this separation by treating every incoming ride, food and parcel request as a generalised geographic payload with its own service constraints. The core of the platform is the Dynamic Multi-Service Feasibility Engine (DMFE), which evaluates every candidate pair of pending requests through a weighted five-factor Compatibility Score covering pickup proximity, route similarity, time-window overlap, vehicle capacity and request priority. Pairs that clear the effective threshold are passed to a Google OR-Tools routing solver, which validates the batch against hard service-level constraints such as the maximum permissible passenger detour and the thermal-decay window of a food order. An adaptive extension of the engine, A-DMFE, adjusts the scoring weights and the acceptance threshold from the operating context instead of holding them fixed. Every batching decision is recorded with a machine-generated explanation, so that a human operator can audit why two requests were combined or rejected. In deterministic closed-loop evaluation over workloads of 50 to 500 concurrent requests against a 60-vehicle fleet, the adaptive engine raised average vehicle utilisation by 3.3 to 5.8 per cent, increased fuel savings by 5.3 to 16.8 per cent and reduced carbon dioxide emissions by up to 45.0 per cent relative to an unbatched baseline, while completing every dispatch wave. The system therefore demonstrates that cross-domain batching of heterogeneous urban transport requests is computationally practical and measurably beneficial.

Keywords – *Unified Mobility Platform, Dynamic Multi-Service Feasibility Engine, Vehicle Routing Problem, Compatibility Score, Google OR-Tools, Explainable Artificial Intelligence, Ride Sharing, Last-Mile Delivery and Sustainable Urban Logistics*

TABLE OF CONTENT

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	x
1	INTRODUCTION	1
	1.1 CORE COMPONENTS OF THE UNIFIED PLATFORM	1
	1.1.1 Unified Request Intake	2
	1.1.2 Dynamic Multi-Service Feasibility Engine	2
	1.1.3 Route Optimisation Layer	2
	1.1.4 Explainable Artificial Intelligence Layer	2
	1.1.5 Adaptive Learning Extension	3
	1.2 TECHNOLOGY STACK	3
	1.2.1 Frontend Development	3
	1.2.2 Backend Development	3
	1.2.3 Database	4
	1.2.4 Optimisation and Routing	4
	1.2.5 Security and Operations	4
	1.3 NEED OF THE PROJECT	4
	1.3.1 Underutilisation of Vehicle Capacity	4
	1.3.2 Avoidable Congestion and Emissions	4
	1.3.3 Fragmented Platform Ecosystem	5
	1.3.4 Opacity of Optimisation Decisions	5
	1.3.5 Rigidity of Fixed Optimisation Parameters	5
	1.4 OBJECTIVE	5
	1.5 ORGANIZATION OF THE REPORT	6
	1.6 SUMMARY	7
2	LITERATURE SURVEY	8
	2.1 INTRODUCTION	8
	2.2 DYNAMIC VEHICLE ROUTING PROBLEMS	8
	2.3 QUANTIFYING THE BENEFITS OF VEHICLE POOLING	8

	2.4 JOINT RIDE-SHARING AND PARCEL DELIVERY	9
	2.5 MATCHING FOR ON-DEMAND FOOD DELIVERY	9
	2.6 SUMMARY	9
3	SYSTEM OVERVIEW	11
	3.1 INTRODUCTION	11
	3.2 EXISTING SYSTEM	11
	3.3 DRAWBACKS OF EXISTING SYSTEM	11
	3.4 PROBLEM STATEMENT	12
	3.5 PROPOSED SYSTEM	12
	3.6 PROPOSED METHODOLOGY	13
	3.7 ADVANTAGES OF PROPOSED SYSTEM	14
	3.8 MODULES OF PROPOSED SYSTEM	15
	3.8.1 Request Management Module	15
	3.8.2 Feasibility Engine Module	15
	3.8.3 Route Optimisation Module	16
	3.8.4 Driver and Fleet Module	16
	3.8.5 Explainability Module	16
	3.8.6 Analytics and Simulation Module	16
	3.9 SUMMARY	17
4	SYSTEM SPECIFICATION	19
	4.1 INTRODUCTION	19
	4.2 HARDWARE SPECIFICATION	19
	4.3 SOFTWARE SPECIFICATION	19
	4.4 SOFTWARE DESCRIPTION	20
	4.4.1 FastAPI	20
	4.4.2 React with Vite	20
	4.4.3 Google OR-Tools	20
	4.4.4 SQLAlchemy and PostgreSQL	21
	4.4.5 Google Maps API	21
	4.5 PROGRAMMING LANGUAGES	21
	4.6 LIBRARIES USED	21
	Google OR-Tools	21
	SQLAlchemy	21

	Recharts and React Leaflet	22
	Pytest	22
	4.7 SUMMARY	22
5	PROPOSED METHODOLOGY	23
	5.1 INTRODUCTION	23
	5.2 MODULE DESCRIPTION	23
	5.2.1 Request Management Module	23
	5.2.2 Feasibility Engine Module	24
	5.2.3 Route Optimisation Module	25
	5.2.4 Driver and Fleet Module	26
	5.2.5 Explainability Module	26
	5.2.6 Analytics and Simulation Module	27
	5.3 SUMMARY	27
6	IMPLEMENTATION AND RESULTS	29
	6.1 IMPLEMENTATION	29
	6.2 EXPERIMENTAL SETUP	33
	6.3 RESULTS AND PERFORMANCE ANALYSIS	33
	6.3.1 Vehicle Utilisation	33
	6.3.2 Fuel and Emission Savings	34
	6.3.3 Batching Rate, Delay and Wave Completion	35
	6.3.4 Computational Cost	36
	6.3.5 Verification	36
	6.3.6 Limitations of the Results	36
	6.4 SUMMARY	37
7	CONCLUSION	38
8	FUTURE ENHANCEMENTS	40
	APPENDIX	42
	REFERENCES	47

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
3.1	Comparison between existing and proposed system	17
6.1	Experimental configuration for the evaluation runs	33
6.2	Average vehicle utilisation, static vs adaptive	34
6.3	Fuel and carbon dioxide savings across workloads	34
6.4	Batching rate, average delay and wave completion	35

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	Architecture of Proposed System	13
3.2	Workflow of the Proposed Methodology	14
3.3	Proposed Modules	15
5.1	Compatibility Scoring and Verification Pipeline	26
5.2	Overall Lifecycle of a Request	27
6.1	System Architecture of the Unified Platform	31
6.2	Administrative Dashboard with Live KPI Cards	31
6.3	Feasibility Engine Panel with Candidate Batches	31
6.4	Explanation Timeline with Score Breakdown	32
6.5	Live Operations Map with Vehicle Markers	32
6.6	Simulation Playback and Scenario Comparison	32

LIST OF ABBREVIATIONS

ACRONYM	ABBREVIATION
API	Application Programming Interface
A-DMFE	Adaptive Dynamic Multi-Service Feasibility Engine
BQS	Batch Quality Score
CO ₂	Carbon Dioxide
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update and Delete
CS	Compatibility Score
DMFE	Dynamic Multi-Service Feasibility Engine
ETA	Estimated Time of Arrival
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
ORM	Object Relational Mapping
OR-Tools	Google Operations Research Tools
RBAC	Role Based Access Control
REST	Representational State Transfer
SLA	Service Level Agreement
SQL	Structured Query Language
TSP	Travelling Salesperson Problem
UI	User Interface
UX	User Experience
VRP	Vehicle Routing Problem
XAI	Explainable Artificial Intelligence

CHAPTER 1

INTRODUCTION

Urban mobility and urban delivery grew up as two separate industries and, in most cities, they are still operated that way. A passenger books a ride on one application, a meal is dispatched from a restaurant through a second application, and a parcel is collected by a third courier network. Each of these platforms maintains its own fleet, its own matching algorithm and its own routing pipeline, and none of them can see the demand held by the others. The consequence is easy to observe on any city road: a vehicle that has just completed a passenger drop returns to its base with an empty cabin and an empty boot, while a parcel that needed to travel along almost the same corridor is assigned to a completely different vehicle. Every such duplicated journey adds a vehicle to the road, burns fuel that need not have been burned, and releases carbon dioxide that the trip did not require.

This project proposes an AI-Powered Unified Mobility and Delivery System that removes the boundary between these service categories. Instead of maintaining three isolated queues, the platform accepts ride, food and parcel requests into a single pending queue and treats each one as a generalised geographic payload defined by a pickup node, a drop node, a time window and a set of service constraints. A Dynamic Multi-Service Feasibility Engine, referred to throughout this report as the DMFE, continuously examines this queue, scores every candidate pair of requests for compatibility, and groups the compatible ones into a batched trip that is served by a single driver and a single vehicle. Combining a passenger ride with a parcel that travels along the same corridor converts an empty return leg into a productive one, and does so without asking the passenger to accept an unreasonable detour.

The engine is deliberately not a black box. Every decision it takes is stored together with the compatibility score that produced it, the individual contribution of each scoring factor, and a natural-language rationale describing why the requests were combined or rejected. This explainability layer allows an operator, an auditor or an examiner to reconstruct the reasoning behind any batch from the recorded state, which is a property that conventional commercial dispatch systems do not offer.

1.1 CORE COMPONENTS OF THE UNIFIED PLATFORM

The platform is built from several core components that work together to convert a stream of independent transport requests into a smaller number of optimised, shared journeys. These components ensure that the system operates smoothly from the moment a request enters the queue to the moment the corresponding trip is dispatched and monitored.

1.1.1 Unified Request Intake

The intake component is the single entry point for every category of demand handled by the platform. A ride request, a food order and a parcel booking are all persisted through the same polymorphic data model, differing only in the constraint block attached to them. A ride carries a maximum detour percentage, a food order carries a thermal-decay window beyond which the meal is considered spoilt, and a parcel carries weight and volume limits. Because all three are stored in the same structure, the scheduling engine can compare them directly without any category-specific special casing, which is precisely the property that makes cross-domain batching possible.

1.1.2 Dynamic Multi-Service Feasibility Engine

The DMFE is the decision-making core of the platform. It runs in two stages. In the first stage a set of inexpensive geometric and temporal pre-checks removes pairs of requests that clearly cannot be served together, such as pairs whose pickup points lie outside the configured radius or whose time windows do not overlap. In the second stage the surviving pairs are scored on five weighted factors and those that clear the effective threshold are promoted to candidate batches. This funnel keeps the expensive combinatorial work confined to a small number of genuinely promising pairs, which is what allows the engine to operate on a continuously refilling queue.

1.1.3 Route Optimisation Layer

Once a candidate batch has been formed, the routing layer must decide the order in which the pickup and drop nodes of the batched requests should be visited, and must confirm that the resulting sequence does not violate the service constraints of any individual request. This is a constrained variant of the travelling salesperson problem and is solved using the Google OR-Tools constraint-programming solver. A distance matrix is constructed for the nodes of the batch, the solver searches for the sequence of minimum total travel cost, and the resulting route is checked against the detour ceiling of each passenger and the freshness window of each food order. If no feasible sequence exists, the batch is rejected and its requests return to the pending queue as individual trips.

1.1.4 Explainable Artificial Intelligence Layer

Combinatorial optimisation is notoriously opaque, and an operator confronted with a batching decision has no way of judging whether it was sound. The explainability layer addresses this by translating the numeric output of the engine into a structured, human-readable rationale. For every batch the system stores the aggregate compatibility score, the signed contribution of each of the five scoring factors, the decision confidence, and the threshold that was in force at the time. An operator inspecting a batch can therefore see not

only that two requests were combined, but that they were combined mainly because their pickup points were nine hundred metres apart and their corridors overlapped by seventy-one per cent.

1.1.5 Adaptive Learning Extension

The static form of the engine applies one fixed set of factor weights and one fixed acceptance threshold to every situation. This is simple but rigid, because a configuration that performs well during a quiet afternoon is not necessarily the configuration that performs well during a dense evening peak. The adaptive extension, A-DMFE, derives an operating context from the current state of the system, comprising demand density, fleet availability and the observed service mix, and generates weights and an effective threshold that suit that context. The adaptive variant is the configuration whose behaviour is measured against the static variant in the experimental evaluation presented in Chapter 6.

1.2 TECHNOLOGY STACK

The technology stack for the project was selected so that the system remains efficient, maintainable and capable of processing a continuously refilling request queue in real time. The stack is divided into the following key components.

1.2.1 Frontend Development

- **React 19 with Vite:** Used to build the administrative and operational single-page application, with route-level code splitting so that heavy dashboards load only after authentication.
- **Tailwind CSS 4:** Provides the utility-first styling system used across every dashboard, keeping the visual language consistent without a large custom stylesheet.
- **Recharts and React Leaflet:** Used for analytics visualisation and for the live map view that renders vehicle and request markers.

1.2.2 Backend Development

- **FastAPI:** An asynchronous Python web framework that exposes the entire platform as a documented REST API, with automatic request validation through Pydantic schemas.
- **SQLAlchemy:** The object relational mapper used to define the polymorphic request, trip, driver and vehicle models and to keep every database access parameterised.

1.2.3 Database

- **PostgreSQL:** The production database, chosen for its reliability under concurrent writes and its strong indexing support.
- **SQLite:** Used transparently in local development so that a developer can run the entire backend without provisioning a database server.

1.2.4 Optimisation and Routing

- **Google OR-Tools:** The constraint-programming and routing library used to solve the batched routing problem and to verify service-level feasibility.
- **Haversine distance model:** Used for fast geospatial pre-filtering and for deterministic evaluation runs in which external map services are deliberately disabled.
- **Google Maps API:** Used in live operation to build realistic travel-time matrices and to render navigation views.

1.2.5 Security and Operations

- **JSON Web Tokens and bcrypt:** Provide stateless authentication and salted password hashing.
- **Security headers and CORS lockdown:** Restrict framing, content-type sniffing and cross-origin access to an explicitly configured list of origins.

1.3 NEED OF THE PROJECT

The need for a unified mobility and delivery platform arises directly from the inefficiencies produced by the current fragmented model. The following subsections describe the specific problems that motivated this work.

1.3.1 Underutilisation of Vehicle Capacity

A vehicle dispatched for a single passenger typically carries one occupant in a cabin designed for four, and an empty boot that could accommodate several parcels. When the trip ends, the vehicle usually returns without any payload at all. Across a working day this deadhead running accounts for a substantial share of total distance travelled, and it produces no revenue for the driver and no service for any customer. A platform that can place a compatible parcel into that same journey converts wasted capacity into completed work.

1.3.2 Avoidable Congestion and Emissions

Every duplicated journey adds one more vehicle to a road network that is already saturated in most Indian cities. Because urban congestion grows sharply once traffic density

approaches capacity, a relatively small reduction in the number of vehicles produces a disproportionately large improvement in average speed. Consolidating overlapping trips therefore reduces not only the fuel consumed by the consolidated vehicles themselves but also the delay imposed on every other road user, and it directly reduces the volume of carbon dioxide released per completed request.

1.3.3 Fragmented Platform Ecosystem

Because ride-hailing, food delivery and parcel logistics are operated by different companies, there is no commercial mechanism through which a passenger trip can be matched with a parcel movement. Each operator optimises only within its own domain and is structurally blind to the demand held by the others. The inefficiency is therefore not a defect of any individual algorithm but a consequence of the way the ecosystem is partitioned, and it can only be addressed by a platform that holds all three categories of demand in one queue.

1.3.4 Opacity of Optimisation Decisions

Where automated batching does exist, it is presented to drivers and customers as a fixed instruction with no accompanying justification. A passenger who is told that the vehicle will make an additional stop has no way of knowing whether the detour was reasonable, and a driver has no basis on which to contest an assignment. This opacity erodes trust in the platform and makes operational faults very difficult to diagnose, because there is no recorded reasoning to inspect after the fact.

1.3.5 Rigidity of Fixed Optimisation Parameters

Dispatch systems are normally tuned once and then left in place. A single fixed acceptance threshold cannot serve both a sparse morning queue, where a permissive threshold is needed to find any batch at all, and a dense evening peak, where a strict threshold is affordable because compatible pairs are plentiful. A system that derives its parameters from the current operating context can maintain a high batching rate across the whole demand cycle rather than only in the conditions for which it was originally tuned.

1.4 OBJECTIVE

The primary objective of this project is to design and implement a unified platform that accepts ride, food and parcel requests into a single scheduling queue and dispatches them through one shared fleet. It aims to develop a Dynamic Multi-Service Feasibility Engine that scores every candidate pair of pending requests on a weighted five-factor compatibility model and forms batches only where all service constraints can be satisfied simultaneously. A further objective is to integrate a constraint-programming routing solver

that determines the optimal visiting sequence for each batch and verifies it against the detour ceiling of every passenger and the freshness window of every food order. The project also seeks to make the optimisation auditable by recording, for every decision, the compatibility score, the contribution of each factor and a natural-language rationale. Finally, the project aims to evaluate the adaptive form of the engine against its static form under controlled, deterministic workloads and to report the resulting differences in vehicle utilisation, fuel consumption, carbon dioxide emission and batching rate honestly, including the cases in which the adaptive form offers no advantage.

1.5 ORGANIZATION OF THE REPORT

The basic organization of the report is as given below

Chapter 1: This chapter introduces the proposed project and explains the need for developing the system. It outlines the main objectives and highlights the cost of operating mobility and delivery networks as separate silos. It also provides a brief overview of how the platform unifies heterogeneous requests and batches them through a feasibility engine.

Chapter 2: This chapter presents a detailed literature survey of existing research related to ride sharing, dynamic vehicle routing, on-demand food delivery and combined passenger-and-goods transport. It reviews the technologies and approaches used in similar systems and identifies the gap that this project addresses.

Chapter 3: This chapter explains the problem addressed by the project and highlights the limitations of existing single-domain dispatch systems. It discusses the challenges involved in combining requests with conflicting constraints and introduces the proposed solution together with its modules and advantages.

Chapter 4: This chapter describes the system specifications required for developing the project. It explains the necessary hardware and software requirements needed to run the system effectively and discusses the development tools, libraries and frameworks used to build the application.

Chapter 5: This chapter explains the methodology used to develop the proposed system. It describes the design and functionality of each module, presents the five-factor compatibility model in detail, and shows how the components interact to produce a dispatched batch.

Chapter 6: This chapter explains the implementation process of the proposed system and the techniques used during development. It presents the results obtained from the deterministic evaluation of the adaptive and static engines and analyses the system performance across four workload levels.

Chapter 7: This chapter summarizes the overall outcome of the project and the key findings obtained from the system implementation. It highlights the effectiveness of the proposed solution in improving fleet utilisation and reducing emissions, and states the limits within which those findings hold.

Chapter 8: This chapter discusses the possible future enhancements and improvements that can be made to the system, including caching of the batch-formation stage, predictive driver pre-positioning and incentive-aligned dynamic pricing.

1.6 SUMMARY

In this chapter, the concept and motivation behind the development of the AI-Powered Unified Mobility and Delivery System have been introduced. The need for a platform that treats ride, food and parcel demand as one scheduling problem rather than three was discussed, along with the project's main objectives and technical approach. The major components, technologies and distinguishing features of the system were highlighted, and an overview of the report organization was provided.

CHAPTER 2

LITERATURE SURVEY

2.1 INTRODUCTION

In this section, we examine the body of research that underpins shared urban transport and combinatorial dispatch. The survey covers four connected strands: classical and dynamic formulations of the vehicle routing problem, the empirical limits of ride pooling, learning-based matching for on-demand food delivery, and the emerging class of systems that carry passengers and goods on the same vehicle. Through a review of prominent research works and implemented models, we aim to identify which parts of the unified-dispatch problem are already well solved, which remain open, and where a feasibility-first engine of the kind proposed in this project can contribute. The survey therefore not only records the current state of the art but also establishes the specific gap that Chapter 3 sets out to address.

2.2 DYNAMIC VEHICLE ROUTING PROBLEMS

Pillac, Gendreau, Gu  ret and Medaglia surveyed the field of dynamic vehicle routing and drew the distinction that organises most subsequent work: a routing problem is dynamic when part of the input is revealed only while the routes are already being executed. Their review classifies problems by whether the newly revealed information is deterministic or stochastic, and shows that re-optimisation approaches, which recompute a solution each time new information arrives, dominate practical deployments because they make no distributional assumptions. Their analysis is directly relevant to this project, because a unified mobility platform is dynamic in exactly this sense: requests arrive continuously and every batching decision must be taken without knowledge of the requests that will arrive a minute later.

2.3 QUANTIFYING THE BENEFITS OF VEHICLE POOLING

Santi, Resta, Szell, Sobolevsky, Strogatz and Ratti introduced the notion of a shareability network and used it to measure, on a corpus of one hundred and fifty million taxi trips, how much of a city's travel demand can be pooled. Their central finding is that the shareability of trips rises very steeply for small increases in the tolerated passenger delay: allowing a delay of only a few minutes makes a large majority of trips shareable, and the cumulative trip length can be cut by roughly forty per cent. This result is important for the present work because it establishes that the gains from pooling are governed less by the sophistication of the algorithm than by the width of the delay tolerance, which is exactly the parameter that the proposed feasibility engine treats as a hard, per-request constraint.

2.4 JOINT RIDE-SHARING AND PARCEL DELIVERY

Zhang, Markos and Yu proposed a joint ride-sharing and parcel delivery strategy for autonomous vehicle fleets, in which a single vehicle serves passenger and parcel demand within the same route. Their work demonstrates that mixing the two payload classes raises fleet utilisation substantially compared with dedicating vehicles to one class, provided that passenger service quality is protected by explicit constraints. Manchella, Haliem, Aggarwal and Bhargava addressed the same combination in PassGoodPool, a distributed reinforcement-learning framework that jointly handles pricing, matching and route planning for passengers and goods. Both studies confirm the economic case for cross-domain batching, but both optimise a learned policy over a combined objective rather than enforcing per-request feasibility before a batch is formed, which is the design choice that distinguishes the engine proposed in this project.

2.5 MATCHING FOR ON-DEMAND FOOD DELIVERY

Chen, Chang and Chang developed an imitation-learning-enhanced iterated matching algorithm for on-demand food delivery, in which a learned model imitates high-quality matching decisions produced offline and then guides an iterative assignment procedure at run time. Liu, Zuo, Zhou, Jia and Xin extended the meal-delivery routing problem to a hybrid fleet of human riders and autonomous vehicles operating in a dynamic environment. Both works highlight the constraint that makes food fundamentally different from passengers and parcels, namely that the value of the payload decays with time in transit, so a route that is optimal in distance may be worthless in service terms. The proposed system adopts this insight by treating the freshness window of a food order as a hard constraint that can veto an otherwise attractive batch.

2.6 SUMMARY

The literature survey establishes four conclusions that shape the design of the proposed system. Pillac and colleagues show that urban dispatch is inherently a dynamic routing problem in which re-optimisation on partial information is the practical approach. Santi and colleagues show that the achievable pooling benefit is large but is governed by the tolerated delay, which justifies making the detour ceiling an explicit per-request parameter rather than a global tuning constant. Zhang, Markos and Yu, together with Manchella and colleagues, demonstrate that carrying passengers and goods on one vehicle is economically worthwhile, while relying on learned policies whose individual decisions cannot easily be audited. Chen and colleagues, together with Liu and colleagues, show that time-decaying payloads such as prepared meals impose constraints that a purely distance-based objective cannot express. Taken together, these works confirm the value of cross-domain batching but leave a clear gap: no reviewed system combines heterogeneous request types under per-

request hard feasibility checks while also recording an auditable rationale for every decision it takes. Closing that gap is the objective of the Dynamic Multi-Service Feasibility Engine presented in the following chapters.

CHAPTER 3

SYSTEM OVERVIEW

3.1 INTRODUCTION

In urban transport, capacity that goes unused is capacity that is lost permanently. A seat that travels empty and a boot that travels empty cannot be recovered at the end of the journey; the fuel has been burned, the road space has been occupied and the emission has been released, and no customer has been served in exchange. Cities across India are currently experiencing simultaneous growth in private vehicle ownership, in app-based ride demand and in doorstep delivery volumes, and because these three streams are served by unconnected platforms the total number of vehicle journeys grows faster than the underlying demand for transport. Congestion, fuel consumption and emissions all rise as a direct consequence. This chapter presents a detailed examination of how transport requests are handled today, identifies the specific inefficiencies that follow from the current arrangement, and proposes a unified system designed to close those gaps.

3.2 EXISTING SYSTEM

The existing arrangement consists of three parallel and mutually invisible dispatch pipelines. A ride-hailing platform maintains a pool of drivers and matches each incoming passenger request to the nearest available vehicle, usually through a proximity heuristic or a variant of the classical assignment algorithm. A food-delivery platform maintains its own rider pool and optimises for the time elapsed between order confirmation and doorstep handover. A parcel courier operates on a separate schedule altogether, aggregating shipments at a hub and dispatching them on fixed routes. Where pooling exists at all, as in shared ride products, it is confined strictly within one service category: two passengers may be pooled with each other, but a passenger is never pooled with a parcel.

Each of these pipelines is internally reasonable. The ride-hailing matcher genuinely does find a near-optimal driver for the passenger it is given, and the delivery matcher genuinely does minimise the food's time in transit. The inefficiency arises at the level above the individual algorithm, because no component of the ecosystem holds all three categories of demand at the same time and therefore no component is in a position to notice that a parcel and a passenger are travelling along the same corridor within the same ten-minute window.

3.3 DRAWBACKS OF EXISTING SYSTEM

The major drawbacks of the existing arrangement follow directly from its partitioned structure. The most significant is deadhead running: because a driver's next

assignment must come from the same category as the previous one, the return leg of a completed trip is almost always empty, and both the seat capacity and the cargo capacity of the vehicle are wasted for the duration of that leg. The second drawback is duplicated vehicle presence, since two requests that overlap geographically are necessarily served by two vehicles, doubling the contribution of that demand to congestion and emissions. The third is the rigidity of fixed matching parameters, which are tuned once for an average operating condition and consequently perform poorly at both extremes of the demand cycle.

A fourth drawback is the absence of any per-request feasibility contract. Pooling decisions in existing shared products are governed by a global tolerance rather than by the specific constraints of the individual requests involved, so a passenger in a hurry and a passenger with time to spare are treated identically. A fifth drawback is opacity: the decision to pool two requests is delivered to the driver and the customer as an instruction with no accompanying justification, which makes disputes impossible to resolve and operational faults very difficult to diagnose after the event.

3.4 PROBLEM STATEMENT

There is an urgent necessity for a unified, automated dispatch platform that can accept heterogeneous transport requests, namely passenger rides, prepared-food orders and physical parcels, into a single scheduling queue and assign them to one shared fleet without violating the distinct temporal and spatial constraints of any individual request. The platform must be able to decide, for any pair of pending requests, whether serving them together is feasible at all, and it must reach that decision quickly enough to keep pace with a queue that refills continuously. It must adapt its acceptance criteria to the prevailing demand density and fleet availability rather than applying one fixed configuration to every operating condition. It must record, for every decision, a rationale that a human operator can inspect and verify. The goal is a dispatch layer that raises vehicle utilisation and reduces emissions while remaining accountable for each grouping it performs.

3.5 PROPOSED SYSTEM

The proposed AI-Powered Unified Mobility and Delivery System is a web-based operational platform that consolidates ride, food and parcel demand into a single dispatch pipeline. Every incoming request, whatever its category, is persisted through the same polymorphic data model and enters one common pending queue. At each scheduling cycle the Dynamic Multi-Service Feasibility Engine examines this queue and evaluates candidate pairs of requests. Pairs that survive a set of inexpensive geometric and temporal pre-checks are scored on five weighted factors, namely pickup proximity, route similarity, time-window overlap, vehicle capacity fit and request priority, and the resulting compatibility score is compared with the effective threshold currently in force.

Pairs that clear the threshold are promoted to candidate batches and handed to the routing layer, which uses the Google OR-Tools constraint-programming solver to determine the optimal visiting sequence over the pickup and drop nodes of the batch. The resulting route is then validated against the hard service constraints of every request it contains: the passenger's detour ceiling, the food order's freshness window and the parcel's weight and volume limits. A batch that fails any of these checks is dissolved and its requests return to the queue as individual trips, so a batch is never dispatched on the strength of its score alone. Successful batches are assigned a driver and a vehicle, and the decision is written to the explanation store together with the score, the factor contributions and the confidence with which it was taken.

Figure 3.1 *Architecture of Proposed System*

3.6 PROPOSED METHODOLOGY

The methodology adopted for the development of the unified platform is centred on a feasibility-first pipeline in which no batch is dispatched until every constraint it touches has been explicitly verified. The system is designed to address the shortcomings of existing single-domain dispatchers by unifying the request model, staging the optimisation work, and making every decision auditable.

The first step involves the construction of a polymorphic persistence layer in which a ride, a food order and a parcel are stored as instances of one request entity distinguished only by a category field and a constraint block. This removes the need for category-specific scheduling code and is the structural prerequisite for cross-domain batching.

The second step is the geospatial and temporal pre-filter. For every candidate pair the engine computes the Haversine distance between pickup points and checks the overlap of the two time windows. Pairs that fail either check are discarded immediately, which keeps the expensive scoring and solving stages confined to a small fraction of the total pair space.

The third step is compatibility scoring. Each surviving pair is evaluated on five factors, each normalised to a value between zero and one and combined through a weight vector to produce a single compatibility score expressed as a percentage. The score is compared with the effective threshold, and the pairs that clear it are ordered by descending score so that the strongest combinations are considered first.

The fourth step is constrained route optimisation. A distance matrix is built over the pickup and drop nodes of the candidate batch and passed to the Google OR-Tools routing solver, which searches for the visiting sequence of minimum total travel cost subject to the precedence requirement that every pickup precedes its own drop. The resulting route is then measured against the per-request service constraints, and only a route that satisfies all of them is accepted.

The fifth step is driver and vehicle selection, in which the accepted batch is matched to an available driver whose vehicle has sufficient seat and cargo capacity for the combined payload and whose current position lies within the dispatch radius of the first pickup node. The final step is explanation capture, in which the compatibility score, the signed contribution of every factor, the effective threshold and the decision confidence are persisted so that the decision can be reconstructed and independently recomputed at any later time.

Figure 3.2 *Workflow of the Proposed Methodology*

3.7 ADVANTAGES OF PROPOSED SYSTEM

The proposed system offers several key advantages over the existing fragmented arrangement. Firstly, unifying ride, food and parcel demand in a single queue exposes batching opportunities that are structurally invisible to any single-domain platform, converting empty return legs into productive ones. Secondly, the two-stage funnel keeps the combinatorial cost manageable, because the expensive solver is invoked only on pairs that have already passed cheap geometric screening. Thirdly, validating each batch against per-request hard constraints guarantees that improvements in fleet utilisation are never obtained

by silently degrading the service promised to an individual customer. Fourthly, the adaptive extension allows the acceptance threshold and the factor weights to follow the demand cycle instead of remaining fixed at a compromise setting. Fifthly, the explanation layer makes every decision auditable, so that a disputed batch can be examined and its score recomputed from recorded state. Finally, consolidating overlapping journeys produces a direct and measurable environmental benefit, since fewer vehicles cover the same demand and less fuel is consumed for each request served.

3.8 MODULES OF PROPOSED SYSTEM

The below Figure 3.3 shows the modules of the proposed system step by step procedures.

Figure 3.3 *Proposed Modules*

The proposed system consists of six primary modules, each designed to fulfil a distinct role in the dispatch process. These modules work collaboratively to ensure rapid, organised and verifiable handling of heterogeneous transport demand. Below is a detailed explanation of each module.

3.8.1 Request Management Module

This module is the intake point for all demand handled by the platform. It exposes the endpoints through which ride, food and parcel requests are created, validated and persisted, and it attaches to each request the constraint block appropriate to its category. It also maintains the pending queue, tracks the age of each waiting request and exposes the queue to the scheduling engine in a form that permits pairwise evaluation. Requests that are dissolved out of a rejected batch are returned to this queue rather than being lost.

3.8.2 Feasibility Engine Module

The Feasibility Engine is the analytical core of the platform. It performs the geospatial and temporal pre-filtering, computes the five-factor compatibility score for every surviving pair, applies the effective threshold and assembles the candidate batches. It

also records, for every pair it examines, whether the pair was accepted or rejected and which factor was chiefly responsible for that outcome, which is the raw material from which the explanation layer later constructs its rationale.

3.8.3 Route Optimisation Module

This module converts an accepted candidate batch into an executable route. It constructs the distance matrix over the batch's nodes, invokes the Google OR-Tools solver to obtain the minimum-cost visiting sequence subject to pickup-before-drop precedence, and then verifies the resulting sequence against the detour ceiling of each passenger and the freshness window of each food order. Where the solver cannot find a feasible sequence, the module reports the failure so that the batch can be dissolved cleanly.

3.8.4 Driver and Fleet Module

The Driver and Fleet Module maintains the state of every driver and every vehicle in the fleet, including availability, current position, seat capacity, cargo capacity, fuel type and mileage. When a batch has been validated, this module selects a driver whose vehicle can carry the combined payload and whose position lies within the dispatch radius of the first pickup, marks the driver and vehicle as busy for the duration of the trip, and releases them on completion. It also contains the recovery logic that releases resources held by trips left in an incomplete state after an unexpected shutdown.

3.8.5 Explainability Module

This module transforms the numeric output of the engine into an auditable record. For every decision it stores the aggregate compatibility score, the signed contribution of each of the five factors, the effective threshold in force at the time, the decision confidence and a natural-language rationale. Because the underlying state is persisted rather than only the narrative, any stored explanation can be independently recomputed and checked for consistency, which is the property that separates a genuine audit trail from a cosmetic one.

3.8.6 Analytics and Simulation Module

The final module provides the operational and experimental views over the platform. It aggregates dispatch outcomes into the key performance indicators shown on the administrative dashboard, including the batching rate, the fuel and carbon dioxide saved and the average distance saved per trip. It also drives the deterministic simulation harness used for the experimental evaluation reported in Chapter 6, allowing a fixed workload to be replayed against different engine configurations under identical seeds so that the comparison between them is meaningful.

Table 3.1 *Comparison between existing and proposed system*

Existing System	Proposed System (Unified DMFE Platform)
Ride, food and parcel demand are held in three separate queues that cannot see one another	All three request categories enter a single polymorphic queue that the scheduler evaluates as one problem
Pooling is confined within a service category; a passenger is never combined with a parcel	Cross-domain batching combines heterogeneous payloads whenever every constraint can be satisfied together
The return leg of a completed trip is almost always empty, wasting both seat and cargo capacity	Empty return legs are filled with compatible payloads, raising utilisation from the same fleet
Matching uses a proximity heuristic with no explicit feasibility contract for the individual request	Every batch is validated against the detour ceiling, freshness window and capacity limit of each request
Acceptance parameters are tuned once and applied to every operating condition	The adaptive engine derives weights and an effective threshold from the current demand and fleet context
Batching decisions are delivered as instructions with no recorded justification	Every decision stores its score, factor contributions, threshold and confidence for later audit
Two overlapping requests occupy two vehicles, doubling their contribution to congestion	Overlapping requests are consolidated, reducing vehicle count for the same served demand
Fuel and emission cost per request is fixed by the one-vehicle-per-request model	Fuel and carbon dioxide saved per request are measured and reported for every dispatched trip
No mechanism exists to replay a day's demand against a different configuration	A deterministic simulation harness replays fixed workloads under identical seeds for controlled comparison
Scaling to new service categories requires a new dispatch pipeline	A new category is added by defining its constraint block; the scheduler is unchanged

3.9 SUMMARY

This chapter analyzed the limitations of the current fragmented dispatch arrangement and emphasized the need for a unified and verifiable alternative. The proposed AI-Powered Unified Mobility and Delivery System places ride, food and parcel demand in one queue, evaluates candidate combinations through a five-factor compatibility model, and dispatches a batch only after a constraint-programming solver has confirmed that every per-request service promise can still be kept. By combining cross-domain batching with

per-request feasibility checks and a persistent explanation record, the system aims to raise fleet utilisation and reduce emissions without degrading individual service quality. The next chapter describes the hardware and software specifications used to build the proposed system in greater detail.

CHAPTER 4

SYSTEM SPECIFICATION

4.1 INTRODUCTION

The effectiveness of a dispatch platform depends greatly on the hardware and software infrastructure that supports it. The AI-Powered Unified Mobility and Delivery System requires a setup capable of running a constraint-programming solver over a continuously refilling request queue, serving an interactive administrative interface, and persisting the request, trip and explanation records that the audit trail depends on. This chapter outlines the essential hardware and software components used in the development, evaluation and deployment of the system.

4.2 HARDWARE SPECIFICATION

Hardware specifications describe the technical setup needed to run and test the system effectively.

System	: Intel Core i5 Processor (11th generation or later).
Hard Disk	: 512 GB Solid State Drive.
RAM	: 8 GB (16 GB recommended for the 500-request evaluation runs).
Display	: 1920 x 1080 resolution or higher for the operational dashboards.
Network	: Broadband connection with access to the mapping and routing services.

4.3 SOFTWARE SPECIFICATION

A software specification document describes the intended purpose, requirements, and nature of a software to be developed.

Operating System	: Windows 11.
Programming Languages	: Python 3.11 (backend) and JavaScript ES2022 (frontend).
Backend Framework	: FastAPI with Uvicorn as the ASGI server.
Frontend Framework	: React 19 built with Vite and styled using Tailwind CSS 4.

Database : PostgreSQL for production and SQLite for local development.

Optimisation Library : Google OR-Tools constraint-programming and routing solver.

Tools : Visual Studio Code, Git, Postman and the Swagger interface at /api/docs.

API Services : Google Maps Distance Matrix and Directions API for live travel-time data.

4.4 SOFTWARE DESCRIPTION

4.4.1 FastAPI

- Used to expose the entire platform as a documented REST API covering authentication, request intake, the feasibility engine, driver management, analytics and playback.
- Validates every incoming payload against a Pydantic schema before it reaches the business logic, so malformed requests are rejected at the boundary.
- Generates interactive OpenAPI documentation automatically, which was used throughout development to exercise endpoints without writing a client.

4.4.2 React with Vite

- Used to build the administrative single-page application containing the dashboard, feasibility-engine panel, explanation timeline, live map and playback views.
- Route-level code splitting through lazy loading defers each heavy dashboard until it is actually opened, keeping the initial payload small.
- Provides the component model on which the reusable status badges, key-performance-indicator cards and modal dialogs of the interface are built.

4.4.3 Google OR-Tools

- Solves the constrained routing problem for each candidate batch, determining the visiting order over the pickup and drop nodes that minimises total travel cost.
- Enforces the precedence requirement that each pickup is visited before its corresponding drop, which is what makes a batched route physically valid.

- Reports infeasibility explicitly when no valid sequence exists, allowing the engine to dissolve the batch rather than dispatch an invalid route.

4.4.4 SQLAlchemy and PostgreSQL

- SQLAlchemy defines the polymorphic request model together with the trip, driver, vehicle, batch and explanation entities, and manages their relationships.
- All database access is parameterised through the object relational mapper, which removes the possibility of structured query language injection across the application.
- PostgreSQL provides the production data store, while an automatically created SQLite database allows the backend to run locally with no server setup.

4.4.5 Google Maps API

- Supplies realistic travel-time matrices for live operation, so that routing decisions reflect actual road conditions rather than straight-line distance.
- Renders the navigation view presented to drivers and the geographic context of the live operations map.
- Is deliberately disabled during the controlled evaluation runs, where a deterministic Haversine distance model is used instead so that results are reproducible.

4.5 PROGRAMMING LANGUAGES

Two programming languages are used in this project. Python is used for the backend, where its scientific and operations-research ecosystem gives direct access to the OR-Tools solver and to the numerical routines used by the scoring model, and where FastAPI provides an asynchronous request-handling layer suited to a service that must respond while a solver is running. JavaScript is used for the frontend, where the React component model supports the interactive dashboards, live map and explanation timeline through which operators interact with the engine. The two halves communicate exclusively over a JSON-based REST interface, so the frontend holds no business logic of its own and every decision reported to the user is one that the backend actually took.

4.6 LIBRARIES USED

Google OR-Tools

OR-Tools is an open-source software suite for combinatorial optimisation developed by Google. In this project its routing solver is used to compute the optimal

visiting sequence for each candidate batch and to detect the cases in which no feasible sequence exists at all. It is the component that converts a compatibility score, which is only an indication of promise, into a verified statement that the batch can actually be served.

SQLAlchemy

SQLAlchemy is the object relational mapper used to define and query the persistence layer. It allows the polymorphic request model, in which ride, food and parcel records share one table and one identifier space, to be expressed cleanly, and it keeps every query parameterised so that user-supplied values are never interpolated into structured query language text.

Recharts and React Leaflet

Recharts is used to render the analytics visualisations on the dashboard, including the utilisation and emission trends across simulation runs. React Leaflet renders the live operations map on which pending requests, active trips and vehicle positions are plotted, giving the operator a geographic view of the state that the engine is reasoning about.

Pytest

Pytest is the testing framework used for the backend suite. It covers the compatibility-scoring functions, the driver-selection logic, the unified scoring path and the adaptive learning engine, and it is the mechanism through which regressions in the engine are detected before they reach the evaluation runs.

4.7 SUMMARY

This chapter provided a clear overview of the technical framework supporting the AI-Powered Unified Mobility and Delivery System, including the essential software and hardware components. It highlighted the use of FastAPI and Python for the backend, React with Vite for the operational interface, Google OR-Tools for constrained route optimisation, and SQLAlchemy with PostgreSQL for persistence. The hardware requirements remain modest, since the computational load is dominated by the batch-formation and solving stages rather than by rendering, which makes the system practical to develop and evaluate on a standard workstation. The next chapter explains how these components are organised into modules and how those modules cooperate to convert a pending request queue into dispatched batches.

CHAPTER 5

PROPOSED METHODOLOGY

5.1 INTRODUCTION

The AI-Powered Unified Mobility and Delivery System is designed to provide an efficient, automated and verifiable method of dispatching heterogeneous transport demand from a single shared fleet. In a dispatch problem the quality of the outcome is decided less by raw computing power than by the order in which decisions are taken, because an expensive optimisation performed on an infeasible combination is wasted work. To address this, the system organises its logic into a sequence of cooperating modules that progressively narrow a large pending queue down to a small set of verified, dispatchable batches. This chapter provides an in-depth explanation of the proposed system's working methodology, including the flow of data between modules, the mathematical form of the compatibility model, and the technologies that support real-time operation.

5.2 MODULE DESCRIPTION

The methodology is based on the integration of six modules that together form a complete dispatch pipeline. Each module consumes the output of the previous stage and either enriches it or eliminates candidates that cannot succeed, so that the computationally expensive stages operate on the smallest possible input. The modules are as follows

- Request Management Module
- Feasibility Engine Module
- Route Optimisation Module
- Driver and Fleet Module
- Explainability Module
- Analytics and Simulation Module

Each module plays a pivotal role in the overall system, ensuring that every stage of the dispatch process is covered, from the arrival of a request in the pending queue to the recording of the rationale behind the decision that dispatched it.

5.2.1 Request Management Module

The Request Management Module is the entry point through which every unit of demand reaches the platform, and it is designed so that the three service categories are indistinguishable to everything downstream of it. Key features of this module include

- **Polymorphic Persistence:** A ride, a food order and a parcel are stored as instances of one request entity, distinguished by a category field and a category-specific constraint block rather than by separate tables.
- **Constraint Attachment:** Each request carries the parameters that define its own service promise, namely the maximum detour percentage for a ride, the thermal-decay window for a food order and the weight and volume limits for a parcel.
- **Queue Maintenance:** The module maintains the pending queue, tracks how long each request has been waiting, and exposes the queue to the scheduler in a form suitable for pairwise evaluation.
- **Safe Re-queueing:** Requests belonging to a batch that fails validation are returned to the pending queue in their original state, so that a rejected batch never causes demand to be lost.

5.2.2 Feasibility Engine Module

The Feasibility Engine is the core of the system, responsible for deciding which pending requests should be served together. It operates in two stages so that expensive work is never spent on hopeless candidates.

- **Geospatial and Temporal Pre-filtering:** For every candidate pair the engine computes the Haversine distance between the two pickup points and checks whether the two service time windows overlap. Pairs that fail either test are discarded before any scoring takes place.
- **Five-Factor Compatibility Scoring:** Each surviving pair is scored on pickup proximity, route similarity, time compatibility, vehicle capacity fit and request priority. Every factor is normalised to the interval from zero to one and combined through a weight vector to produce a single compatibility score.
- **Threshold Application:** The compatibility score is compared with the effective threshold in force at that moment. Pairs at or above the threshold are promoted to candidate batches; those below it are recorded as rejected together with the factor chiefly responsible.
- **Ordered Promotion:** Accepted pairs are sorted in descending order of score, so that the strongest combinations claim fleet capacity first and weaker ones are considered only with what remains.

The compatibility score is computed as a weighted linear combination of the five normalised factors, as shown below, where every factor lies between zero and one and the weights sum to unity.

$$CS = w1 \cdot Pickup + w2 \cdot Route + w3 \cdot Time + w4 \cdot Capacity + w5 \cdot Priority$$

The default weight vector used by the static engine assigns thirty per cent to pickup proximity, twenty-five per cent to route similarity, twenty per cent to time compatibility, fifteen per cent to vehicle capacity fit and ten per cent to request priority. Pickup proximity carries the largest weight because the distance between two pickup points is the single strongest predictor of whether a shared route can be constructed at all, while priority carries the smallest because it expresses urgency rather than physical feasibility. In the adaptive configuration this vector is not fixed; it is regenerated from the current operating context so that the engine can favour proximity in a sparse queue and route similarity in a dense one.

5.2.3 Route Optimisation Module

Once a candidate batch has been accepted by the scoring stage, the Route Optimisation Module must establish whether it can actually be served, because a high compatibility score indicates promise rather than feasibility. The key features of this module include

- **Distance Matrix Construction:** A matrix of pairwise travel costs is built over all pickup and drop nodes belonging to the batch, using live travel times in production and a deterministic Haversine model during controlled evaluation.
- **Constrained Sequencing:** The Google OR-Tools routing solver searches for the visiting sequence of minimum total travel cost, subject to the precedence constraint that every pickup is visited before its own drop.
- **Service-Level Verification:** The resulting route is measured against the detour ceiling of every passenger in the batch and the freshness window of every food order, so that no customer's individual promise is broken in order to improve an aggregate metric.
- **Explicit Infeasibility Reporting:** Where the solver cannot produce a sequence that satisfies every constraint, the module reports failure so that the batch is dissolved and its requests are returned to the queue as individual trips.

Figure 5.1 *Compatibility Scoring and Verification Pipeline*

5.2.4 Driver and Fleet Module

After a batch has been verified as routable, the Driver and Fleet Module is responsible for allocating the physical resources that will execute it. The key features of this module include

- **Capacity Matching:** The module selects a vehicle whose seat capacity accommodates the passengers in the batch and whose cargo capacity accommodates the combined weight and volume of the food and parcel payloads.
- **Proximity Gating:** Only drivers whose current position lies within the configured dispatch radius of the first pickup node are considered, so that the approach leg does not consume the saving that batching created.
- **State Transition:** The selected driver and vehicle are marked as busy for the duration of the trip and released automatically on completion, keeping the availability view consistent with reality.
- **Stale Resource Recovery:** Trips left in an incomplete state by an unexpected shutdown are released after a configured interval, which prevents a fleet from being permanently locked out by resources that were never returned.

5.2.5 Explainability Module

In addition to producing dispatch decisions, the system is required to be able to justify them. The Explainability Module converts the numeric output of the engine into a record that a human operator can inspect and verify. The main objectives of this module include

- **Factor Attribution:** For every decision the module stores the signed contribution of each of the five factors, so that an operator can see which factor carried the batch and which held it back.
- **Decision Context Capture:** The effective threshold in force at the moment of the decision, the decision confidence and the batch quality score are persisted alongside the outcome.
- **Natural-Language Rationale:** The stored numeric state is rendered into a readable explanation, for example that two requests were combined because their pickup points lay nine hundred metres apart and their corridors overlapped substantially.

- **Independent Recomputation:** Because the underlying state is persisted rather than only the narrative, any stored explanation can be recomputed from the recorded values and checked for consistency, which is what makes the audit trail meaningful rather than decorative.

Figure 5.2 *Overall Lifecycle of a Request*

5.2.6 Analytics and Simulation Module

The final module provides the operational and experimental views over the platform. It aggregates dispatch outcomes into the key performance indicators presented on the administrative dashboard, including the batching rate, the volume of fuel saved, the mass of carbon dioxide avoided and the average distance saved per trip, and it enforces the rule that a quantity which cannot be derived honestly from the recorded data is omitted from the interface rather than approximated. The module also drives the deterministic simulation harness used in Chapter 6, which replays a fixed workload against different engine configurations under identical random seeds so that any observed difference is attributable to the engine rather than to the demand pattern.

5.3 SUMMARY

This chapter provided a detailed overview of the proposed methodology for the AI-Powered Unified Mobility and Delivery System. The system is designed to streamline the entire dispatch process, from the arrival of a heterogeneous request in the pending queue through compatibility scoring, constrained route optimisation and fleet allocation, to the recording of an auditable rationale for the decision taken. The five-factor compatibility model was presented together with the reasoning behind its weight distribution, and the role of the constraint-programming solver in converting a promising score into a verified batch was explained. Each module works in sequence to eliminate infeasible candidates as early and as cheaply as possible, so that the expensive stages operate only on combinations that

can genuinely succeed. The next chapter describes how these modules were implemented and presents the experimental results obtained from the deterministic evaluation of the system.

CHAPTER 6

IMPLEMENTATION AND RESULTS

This chapter provides an in-depth look into the implementation process of the AI-Powered Unified Mobility and Delivery System, covering the key steps involved in building the platform, integrating the necessary services, and evaluating the results. The development of the system is broken down into several phases, including the construction of the polymorphic persistence layer, the implementation of the five-factor feasibility engine, the integration of the constraint-programming routing solver, the capture of explanation records, and finally the controlled experimental comparison between the static and adaptive forms of the engine. The chapter concludes with a detailed analysis of the results obtained and an honest account of the conditions under which those results hold.

6.1 IMPLEMENTATION

The implementation of the system involved several critical steps, ensuring a correct and traceable flow of data from the arrival of a request to the dispatch of a verified batch. Below are the primary implementation steps.

- **Building the Polymorphic Persistence Layer**

The first task was to design a data model in which a passenger ride, a prepared-food order and a physical parcel are stored as instances of a single request entity. Each record carries a category field and a constraint block holding the parameters specific to that category, while all records share the same identifier space, the same lifecycle states and the same relationship to trips and batches. This decision is what makes cross-domain batching possible at all, because the scheduling engine can compare any two requests directly without category-specific branching. Alongside the request entity, the schema defines provider, vehicle, driver, trip, batch and analysis-run entities, all mapped through SQLAlchemy so that every query is parameterised and no user-supplied value is ever interpolated into structured query language text.

- **Implementing the Five-Factor Feasibility Engine**

The scoring module was implemented as a set of pure functions with no database access and no external dependencies, each returning a value between zero and one for one of the five factors. This isolation was deliberate, because it allows every factor to be unit tested independently and allows the weighted aggregate to be recomputed from stored state during an audit. The aggregation function accepts the factor scores and the weight vector, verifies that the weights sum to unity, and returns the compatibility score as a percentage. A

separate decision layer applies the effective threshold, orders the accepted pairs by descending score and assembles the candidate batches.

- **Integrating the Google OR-Tools Routing Solver**

Each candidate batch is converted into a routing problem by building a distance matrix over its pickup and drop nodes and passing that matrix to the OR-Tools routing solver. The solver is configured with a precedence constraint requiring that each pickup is visited before its own drop, and it returns either the minimum-cost visiting sequence or an explicit statement that no feasible sequence exists. The returned route is then measured against the detour ceiling of every passenger and the freshness window of every food order in the batch. Only a route that satisfies all of these constraints is accepted; any other outcome causes the batch to be dissolved and its requests to be returned to the pending queue unchanged.

- **Implementing Driver Selection and Fleet State Management**

Driver selection matches a verified batch to a driver whose vehicle can carry the combined passenger and cargo payload and whose current position lies within the dispatch radius of the first pickup node. The selected driver and vehicle are marked busy for the duration of the trip and released on completion. During development it was found that trips left in a planned or active state by an unexpected server shutdown kept their drivers permanently busy, which starved the availability gate and caused the engine to reject every subsequent batch. A recovery routine that releases such trips at startup was therefore added, and it forms part of the application lifespan handler.

- **Capturing Explanation Records**

Every decision taken by the engine, whether it accepted or rejected a pair, is written to the explanation store together with the aggregate compatibility score, the signed contribution of each of the five factors, the effective threshold in force at that moment, the decision confidence and the batch quality score. Because the numeric state is persisted rather than only a narrative sentence, a stored explanation can be independently recomputed and compared against the value that was recorded. In audited runs the recomputed compatibility scores matched the stored values closely, confirming that the attribution is reproducible rather than cosmetic.

- **Building the Operational Interface**

The administrative interface was implemented as a React single-page application communicating with the backend exclusively over the REST interface. It presents the live dashboard with its key performance indicators, the feasibility-engine panel showing candidate batches and rejected pairs, the explanation timeline with per-factor breakdowns, the live operations map, the analytics views and the simulation playback screens. Route-

level lazy loading defers each heavy dashboard until it is opened, and a protected-route wrapper enforces role-based access on the client side to match the dependency-injected authorisation applied on the server.

Figure 6.1 *System Architecture of the Unified Platform*

Figure 6.2 *Administrative Dashboard with Live KPI Cards*

Figure 6.3 *Feasibility Engine Panel with Candidate Batches*

Figure 6.4 *Explanation Timeline with Score Breakdown*

Figure 6.5 *Live Operations Map with Vehicle Markers*

Figure 6.6 *Simulation Playback and Scenario Comparison*

6.2 EXPERIMENTAL SETUP

To evaluate the system, a deterministic simulation harness was used in place of live traffic, so that the adaptive and static forms of the engine could be compared under identical conditions. Each experiment generates a fixed workload of heterogeneous requests over an urban grid, assigns them to a fixed fleet, and runs the complete dispatch pipeline from scoring through routing to driver allocation. Because every run is seeded, replaying the same workload against a different engine configuration isolates the effect of the engine itself from the effect of the demand pattern.

Four workload levels were used, namely fifty, one hundred, two hundred and fifty, and five hundred concurrent requests, against a fleet of sixty vehicles and sixty drivers. The service mix was held at forty per cent ride, forty per cent food and twenty per cent parcel throughout. Distances were computed with a deterministic Haversine model and the external mapping service was disabled, so that no network variability could enter the measurements. A fresh database schema was created for every run. Each configuration was executed as a single dispatch pass and also as a simulated multi-wave full-day operation, so that the behaviour of demand exceeding instantaneous fleet capacity could be observed.

Table 6.1 *Experimental configuration for the evaluation runs*

Parameter	Value
Workload levels	50, 100, 250 and 500 concurrent requests
Fleet size	60 vehicles and 60 drivers
Service mix	40 per cent ride, 40 per cent food, 20 per cent parcel
Repetitions	5 seeds at each workload up to 250; 3 seeds at 500
Distance model	Deterministic Haversine (external mapping disabled)
Database	Fresh schema created for every run
Execution modes	Single dispatch pass and simulated multi-wave full-day operation

6.3 RESULTS AND PERFORMANCE ANALYSIS

6.3.1 Vehicle Utilisation

Average vehicle utilisation measures the proportion of the deployed fleet's capacity that is actually carrying a payload. It is the metric most directly connected to the purpose of the system, since the entire justification for cross-domain batching is that capacity which

would otherwise travel empty is put to use. Table 6.2 reports utilisation for both engine configurations at each workload level.

Table 6.2 *Average vehicle utilisation, static vs adaptive*

Workload (requests)	Static DMFE (%)	Adaptive A-DMFE (%)	Change (%)
50	74.28	77.33	+4.1
100	75.60	78.49	+3.8
250	75.31	77.81	+3.3
500	76.36	80.81	+5.8

The adaptive engine achieved higher utilisation at every workload, with the improvement ranging from 3.3 per cent at two hundred and fifty requests to 5.8 per cent at five hundred. The gain is consistent in direction across all four levels, which indicates that deriving the acceptance threshold and the weight vector from the operating context rather than fixing them in advance allows the engine to find batches that the static configuration declines.

6.3.2 Fuel and Emission Savings

Fuel saved and carbon dioxide avoided are computed for each dispatched trip by comparing the distance actually travelled by the batched route against the sum of the distances that the constituent requests would have required if served individually. Table 6.3 reports the totals for each workload, together with the percentage reduction in carbon dioxide relative to an unbatched baseline in which every request is served by its own vehicle.

Table 6.3 *Fuel and carbon dioxide savings across workloads*

Workload	Fuel saved, static (L)	Fuel saved, adaptive (L)	CO2 saved, static (kg)	CO2 saved, adaptive (kg)	CO2 reduction, adaptive (%)
50	3.40	3.97	7.78	9.07	20.1
100	15.28	16.09	35.15	36.98	29.4
250	30.86	34.94	70.99	80.38	44.9
500	32.01	35.14	73.56	80.81	45.0

The adaptive engine saved more fuel and avoided more carbon dioxide than the static engine at every workload, with the fuel improvement ranging from 5.3 per cent at one hundred requests to 16.8 per cent at fifty. Measured against the unbatched baseline, the adaptive engine avoided up to 45.0 per cent of the carbon dioxide that individual dispatch would have produced. The environmental benefit grows with workload density, which is the expected behaviour: a denser queue contains more geographically compatible pairs, so a larger share of the demand can be consolidated.

6.3.3 Batching Rate, Delay and Wave Completion

The batching rate is the proportion of dispatched trips that carry more than one request, and is therefore a direct measure of how often the engine succeeds in finding a viable combination. Average delay is the additional travel time imposed on a request by the batched route relative to a direct journey, and is the cost that customers bear in exchange for the efficiency gain. Wave completion records whether all demand was eventually served across the simulated full-day operation.

Table 6.4 *Batching rate, average delay and wave completion*

Workload	Batching rate, static (%)	Batching rate, adaptive (%)	Avg delay, static (min)	Avg delay, adaptive (min)	Wave completion (%)
50	42.9	44.1	4.41	4.53	100
100	69.5	71.9	5.44	5.41	100
250	83.3	83.3	4.72	4.47	100
500	83.3	83.3	4.34	4.48	100

Two observations follow from Table 6.4. First, the adaptive advantage in batching rate appears only at the lower workloads, where it adds between 2.8 and 3.5 per cent, and disappears entirely at two hundred and fifty requests and above, where both configurations reach 83.3 per cent. This is because at high density the static engine already saturates the available batching opportunity, leaving the adaptive engine nothing further to find. Second, average delay remains between 4.3 and 5.4 minutes in both configurations at every workload, so the efficiency gains reported above were not obtained by degrading service quality. Wave completion is one hundred per cent in every configuration, meaning that demand exceeding instantaneous fleet capacity was queued and fully served by subsequent dispatch waves rather than being lost.

6.3.4 Computational Cost

The adaptive configuration is not free. Profiling the pipeline showed that the additional context derivation and threshold computation raise processing cost from approximately 9.5 per cent per request at the fifty-request workload to approximately 84.9 per cent at five hundred, with total pipeline wall time rising from 0.92 seconds to 7.36 seconds across that range. Stage-level profiling further showed that the dominant cost is batch formation rather than route optimisation: at the five-hundred-request workload, batch formation accounted for 57.9 per cent of adaptive pipeline wall time against 31.5 per cent for the static configuration, while route optimisation never exceeded 7.7 per cent. This locates the bottleneck precisely, and it is the finding that motivates the caching and parallelisation work proposed in Chapter 8.

6.3.5 Verification

The engine was verified through a layered testing strategy before the evaluation runs were performed. The dedicated verification suite for the adaptive engine passed all fifty-three of its checks. The application programming interface smoke tests returned successful responses from every endpoint, and configuration changes made through the interface were confirmed to be recorded in the audit log. Database integrity was verified on each of the evaluation databases, and a regression test confirmed that the static configuration continued to behave exactly as it had before the adaptive extension was introduced. Over one hundred and twenty isolated solver invocations were executed to confirm that the OR-Tools integration is stable and leaks no resources, and the relaxation path taken when no solution exists was exercised explicitly.

6.3.6 Limitations of the Results

Three qualifications must be attached to the figures reported above. First, the evaluation is a deterministic simulation in which repeated seeds vary only the random number stream, so the reported means are descriptive summaries rather than the outcome of statistical significance testing. Second, at the two hundred and fifty and five hundred request workloads a sixty-vehicle fleet cannot dispatch all demand in a single pass, so the requests served in that pass are the most batchable subset; the comparison at those workloads is therefore not entirely like for like, and the strongest claims rest on the fifty and one hundred request cases. Third, the rates reported here are dispatch rates, meaning that a request is counted as served once it has been assigned to a driver and vehicle. Trip execution itself is simulated, so completion is not an independently observed outcome. Stating these limits explicitly is what allows the remaining findings to be relied upon.

6.4 SUMMARY

This chapter provided a comprehensive overview of the implementation process for the AI-Powered Unified Mobility and Delivery System and presented the results of its experimental evaluation.

Key points covered include

- The construction of a polymorphic persistence layer in which ride, food and parcel requests share one entity and one lifecycle.
- The implementation of the five-factor compatibility engine as isolated, independently testable scoring functions.
- The integration of the Google OR-Tools solver to verify each candidate batch against per-request service constraints before dispatch.
- The capture of explanation records containing scores, factor contributions, thresholds and confidence, enabling independent recomputation.
- The controlled comparison of the adaptive and static engines across four workload levels under identical seeds.

The evaluation showed that the adaptive engine raised average vehicle utilisation by between 3.3 and 5.8 per cent, increased fuel savings by between 5.3 and 16.8 per cent, and avoided up to 45.0 per cent of the carbon dioxide that unbatched dispatch would have produced, while holding average delay between 4.3 and 5.4 minutes and completing every dispatch wave. The advantage in batching rate is confined to low and moderate workloads, and the adaptive configuration carries a real computational cost concentrated in the batch-formation stage. Reporting both the gains and their boundaries is what makes the evaluation usable as evidence.

CHAPTER 7

CONCLUSION

The proposed AI-Powered Unified Mobility and Delivery System successfully addresses the structural inefficiency that arises when passenger mobility, food delivery and parcel logistics are operated as unconnected networks. By persisting all three categories of demand through a single polymorphic request model and evaluating them in one common scheduling queue, the platform exposes batching opportunities that no single-domain dispatcher is able to see. The Dynamic Multi-Service Feasibility Engine scores every candidate pair on pickup proximity, route similarity, time compatibility, vehicle capacity fit and request priority, and promotes only those pairs that clear the effective threshold, so that the expensive combinatorial stage operates on a small and genuinely promising set of candidates.

The decisive design choice in this work is that a high compatibility score is treated as a hypothesis rather than as a conclusion. Every candidate batch is passed to a Google OR-Tools constraint-programming solver, which determines the optimal visiting sequence over the batch's pickup and drop nodes and verifies that sequence against the detour ceiling of each passenger, the freshness window of each food order and the capacity limits of the assigned vehicle. A batch that fails any of these checks is dissolved and its requests are returned to the queue as individual trips. As a result, the efficiency reported by the platform is never obtained by quietly breaking a promise made to an individual customer.

The experimental evaluation confirmed the value of this approach. Across workloads of fifty to five hundred concurrent requests served by a sixty-vehicle fleet, the adaptive engine raised average vehicle utilisation by between 3.3 and 5.8 per cent over the static configuration, increased fuel savings by between 5.3 and 16.8 per cent, and avoided up to 45.0 per cent of the carbon dioxide that unbatched dispatch would have produced, while holding average delay between 4.3 and 5.4 minutes and completing every dispatch wave. The advantage in batching rate is real at low and moderate demand and disappears at high density, where the static engine already saturates the available opportunity, and the adaptive configuration carries a measurable computational cost concentrated in the batch-formation stage. Reporting these boundaries alongside the gains is what makes the result usable as evidence rather than as a claim.

Finally, the explainability layer distinguishes this system from conventional dispatch software. Because the compatibility score, the signed contribution of each factor, the effective threshold and the decision confidence are all persisted rather than discarded, any batching decision can be reconstructed and independently recomputed long after it was taken. In audited runs the recomputed scores matched the stored values, establishing that

the audit trail reflects the engine's actual reasoning. Taken together, the unified request model, the feasibility-first pipeline and the persistent explanation record demonstrate that cross-domain batching of heterogeneous urban transport demand is computationally practical, environmentally beneficial and operationally accountable.

CHAPTER 8

FUTURE ENHANCEMENTS

The most immediate enhancement follows directly from the profiling results reported in Chapter 6. Batch formation, not route optimisation, dominates the pipeline at high workloads, accounting for close to sixty per cent of wall time when five hundred concurrent requests are processed, because every candidate pair is rescored from scratch on each dispatch cycle. Introducing a cache for the distance matrix and the geometric components of the compatibility score, so that a pair whose inputs have not changed is not recomputed, would remove a large part of this cost. Parallelising the scoring stage across worker processes would reduce it further, since the pairwise evaluations are independent of one another and therefore embarrassingly parallel.

A second enhancement is predictive driver pre-positioning. At present the platform reacts to demand once a request has been created, so a batch can only be formed if a suitable driver already happens to be near the pickup corridor. A machine learning model trained on historical request density by time of day and geographic cell could forecast where demand is likely to appear and guide idle drivers towards those areas before the requests arrive. This would raise the proportion of candidate batches that pass the proximity gate and would shorten the approach leg that currently consumes part of the saving that batching creates.

A third direction is incentive-aligned dynamic pricing. The present system treats the passenger's willingness to accept a detour as a fixed constraint, whereas in practice that willingness varies between customers and between journeys. Offering a fare reduction to passengers who opt in to allow a parcel to share their vehicle, and adjusting that reduction according to the current scarcity of compatible capacity, would convert a rigid constraint into a negotiable one and would increase the pool of batchable demand without any change to the underlying engine.

A fourth enhancement concerns the evaluation itself. The strongest results reported in this work rest on the fifty and one hundred request workloads, because at higher volumes a sixty-vehicle fleet cannot serve all demand in a single pass and the served subset is biased towards the most batchable requests. Repeating the experiments with the fleet scaled in proportion to demand would remove this asymmetry and would establish whether the adaptive advantage holds at every fleet size. Extending the harness so that trips are executed rather than only dispatched would also allow genuine completion rates to be measured instead of dispatch rates.

Finally, the operational layer can be strengthened by replacing the current polling-based position updates with a persistent WebSocket channel backed by Redis, which would

allow live tracking to scale to a realistic fleet without placing repeated load on the REST interface. Together, these enhancements would move the platform from a validated research prototype towards a system capable of operating continuously at city scale.

APPENDIX

SOURCE CODE

scoring.py — Five-Factor Compatibility Scoring Module

```
"""
DMFE Scoring Module - Core Engine
=====
Pure scoring functions providing exactly the five factors required by the
weighted Compatibility Score formula:

    CS = w1*Pickup + w2*Route + w3*Time + w4*Capacity + w5*Priority

Every factor function returns a float in [0.0, 1.0] where 1.0 is the most
compatible outcome. No database access, no routing engine, no external
APIs - only arithmetic on coordinates, vectors and timestamps.
"""

from __future__ import annotations
from datetime import datetime
from typing import Dict, List, Tuple

from app.dmfe.score_engine import (
    pickup_distance_score, destination_similarity_score,
    route_overlap_score, time_window_score,
    priority_score as _priority_score,
    vehicle_capacity_score as _capacity_score,
)

# Configurable default weights (w1..w5) - must sum to 1.0
DEFAULT_WEIGHTS: Dict[str, float] = {
    "pickup": 0.30, # w1 - Pickup Proximity Score
    "route": 0.25, # w2 - Route Similarity Score
    "time": 0.20, # w3 - Time Compatibility Score
    "capacity": 0.15, # w4 - Vehicle Capacity Score
    "priority": 0.10, # w5 - Priority Score
}

FACTOR_KEYS: Tuple[str, ...] = tuple(DEFAULT_WEIGHTS.keys())

# 1. Pickup Proximity Score
def pickup_proximity_score(
    lat1: float, lng1: float,
    lat2: float, lng2: float,
    max_radius_km: float = 5.0,
) -> Tuple[float, float]:
    """Score based on the real-world distance between two pickup
    locations. Closer pickups -> higher score.
    Returns (score, distance_m)."""
    return pickup_distance_score(lat1, lng1, lat2, lng2, max_radius_km)

# 2. Route Similarity Score
def route_similarity_score(
    pickup1_lat, pickup1_lng, drop1_lat, drop1_lng,
```

```

    pickup2_lat, pickup2_lng, drop2_lat, drop2_lng,
) -> Tuple[float, Dict[str, float]]:
    """Average of two complementary measures:
    - direction_similarity: cosine similarity of the two trip vectors
    - overlap_score: fraction of the combined trip that is shared
    """
    direction = destination_similarity_score(
        pickup1_lat, pickup1_lng, drop1_lat, drop1_lng,
        pickup2_lat, pickup2_lng, drop2_lat, drop2_lng,
    )
    overlap, label = route_overlap_score(
        pickup1_lat, pickup1_lng, drop1_lat, drop1_lng,
        pickup2_lat, pickup2_lng, drop2_lat, drop2_lng,
    )
    score = (direction + overlap) / 2.0
    return round(score, 4), {
        "direction_similarity": direction,
        "overlap_score": overlap,
        "overlap_label": label,
    }

# 3. Time Compatibility Score
def time_compatibility_score(
    ts1: datetime, ts2: datetime, max_delay_min: float = 20.0,
) -> Tuple[float, float]:
    """Requests within the same time window score 1.0; requests beyond
    max_delay_min score 0.0 (linear decay in between)."""
    return time_window_score(ts1, ts2, max_delay_min)

# 4. Vehicle Capacity Score
def vehicle_capacity_score(
    total_demand: int, total_weight_kg: float,
    max_capacity: int = 6, max_weight_kg: float = 100.0,
) -> Tuple[float, float, str]:
    """Score based on whether the combined requests fit in a single
    vehicle. Returns (score, utilization_pct, note)."""
    score, note = _capacity_score(
        total_demand, total_weight_kg,
        max_capacity=max_capacity, max_weight_kg=max_weight_kg,
    )
    demand_ratio = total_demand / max(max_capacity, 1)
    weight_ratio = total_weight_kg / max(max_weight_kg, 1.0)
    utilization = round(max(demand_ratio, weight_ratio) * 100.0, 1)
    return round(score, 4), utilization, note

# 5. Priority Score
def priority_score(priorities: List[str]) -> Tuple[float, str]:
    """Weighted average priority across all requests in the candidate
    group. Returns (score, dominant_priority_label)."""
    return _priority_score(priorities)

# Weighted Compatibility Score (CS)
def weighted_compatibility_score(
    weights: Dict[str, float],

```

```

    factor_scores: Dict[str, float],
) -> float:
    """CS = w1*Pickup + w2*Route + w3*Time + w4*Capacity + w5*Priority

    Accepts factor scores in [0, 1] and returns the weighted aggregate
    mapped to a 0-100 percentage, rounded to 1 decimal place.
    Weights that sum to less than 1.0 are normalised so the result is
    always a true percentage of the theoretical maximum."""
    total_w = sum(weights.get(k, 0.0) for k in FACTOR_KEYS)
    if total_w <= 0:
        return 0.0
    cs = sum(
        weights.get(k, 0.0) * factor_scores.get(k, 0.0)
        for k in FACTOR_KEYS
    )
    return round((cs / total_w) * 100.0, 1)

```

main.py — FastAPI Application Entry Point

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.api.routes import auth, providers, dashboard, orchestration
from app.api.routes import simulation, xai, notifications, drivers
from app.api.routes import config, playback, dmfe_v2, dmfe_engine
from app.core.config import settings
from contextlib import asynccontextmanager
from app.db.database import engine, Base, SessionLocal
from app.db.models import User
from app.core.security import get_password_hash
from app.core.middleware import (
    SecurityHeadersMiddleware, RequestLoggingMiddleware,
)
from app.db.database import sync_schema_columns

```

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    sync_schema_columns()
    Base.metadata.create_all(bind=engine)
    db = SessionLocal()
    try:
        admin = db.query(User).filter(
            User.email == "admin@aiorch.com").first()
        if not admin:
            admin = User(
                email="admin@aiorch.com",
                full_name="Platform Admin",
                password_hash=get_password_hash("admin123"),
                role="Admin",
            )
            db.add(admin)
            db.commit()
    try:
        from app.services.driver_service import driver_service
        driver_service.seed_initial_data_if_needed(db)

```

```

    except Exception:
        import logging
        logging.getLogger(__name__).exception(
            "driver/vehicle seed failed")
    finally:
        db.close()

# Release trips stuck in Planned/Active from a previous server run.
# Without this, their drivers/vehicles stay Busy forever and the
# DMFE driver-availability gate rejects every batch.
try:
    from app.dmfe.driver_selection import complete_stale_trips
    db = SessionLocal()
    try:
        complete_stale_trips(db, max_age_min=30.0)
    finally:
        db.close()
except Exception as exc:
    import logging
    logging.getLogger(__name__).warning(
        "stale-trip cleanup failed: %s", exc)
yield

app = FastAPI(
    lifespan=lifespan,
    title=settings.PROJECT_NAME,
    description="AI-Powered Unified Mobility and Delivery System",
    version="1.0.0",
    openapi_url="/api/openapi.json",
    docs_url="/api/docs",
    redoc_url="/api/redoc",
)

app.add_middleware(SecurityHeadersMiddleware)
app.add_middleware(RequestLoggingMiddleware)

origins = [o.strip() for o in settings.ALLOWED_ORIGINS.split(",")]
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

App.jsx — React Application Shell and Routing

```

import React, { Suspense, lazy } from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate }
  from 'react-router-dom';
import { Toaster } from 'react-hot-toast';
import { AuthProvider } from './context/AuthContext';
import AdminLayout from './components/AdminLayout';
import ProtectedRoute from './components/ProtectedRoute';

```

```

import AnimatedBackground from './components/ui/AnimatedBackground';

const Login = lazy(() => import('./pages/Login'));
const Dashboard = lazy(() => import('./pages/Dashboard'));
const ProviderManagement =
  lazy(() => import('./pages/ProviderManagement'));
const DriverDashboard = lazy(() => import('./pages/DriverDashboard'));
const AnalyticsDashboard =
  lazy(() => import('./pages/AnalyticsDashboard'));
const ExplanationDashboard =
  lazy(() => import('./pages/ExplanationDashboard'));
const DMFEDashboard = lazy(() => import('./pages/DMFEDashboard'));
const LiveSimulationMap =
  lazy(() => import('./pages/LiveSimulationMap'));

function App() {
  return (
    <AuthProvider>
      <AnimatedBackground />
      <Router>
        <Toaster position="top-right" />
        <Suspense fallback={<PageLoader />}>
          <Routes>
            <Route path="/login" element={<Login />} />
            <Route path="/" element={
              <ProtectedRoute><AdminLayout /></ProtectedRoute>>
              <Route index
                element={<Navigate to="/dashboard" replace />} />
              <Route path="dashboard" element={<Dashboard />} />
              <Route path="providers"
                element={<ProviderManagement />} />
              <Route path="drivers" element={<DriverDashboard />} />
              <Route path="analytics"
                element={<AnalyticsDashboard />} />
              <Route path="xai"
                element={<ExplanationDashboard />} />
              <Route path="dmfe" element={<DMFEDashboard />} />
              <Route path="live-map"
                element={<LiveSimulationMap />} />
            </Route>
            <Route path="*"
              element={<Navigate to="/dashboard" replace />} />
          </Routes>
        </Suspense>
      </Router>
    </AuthProvider>
  );
}

export default App;

```

REFERENCES

- [1] **A. L. Medaglia**, V. Pillac, M. Gendreau and C. Gu  ret, “A review of dynamic vehicle routing problems,” *Eur. J. Oper. Res.*, vol. 225, no. 1, pp. 1–11, 2013.
- [2] **B. Bhargava**, K. Manchella, M. Haliem and V. Aggarwal, “PassGoodPool: Joint passengers and goods fleet management with reinforcement learning aided pricing, matching, and route planning,” *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 20854–20868, Nov. 2022.
- [3] **C. Ratti**, P. Santi, G. Resta, M. Szell, S. Sobolevsky and S. H. Strogatz, “Quantifying the benefits of vehicle pooling with shareability networks,” *Proc. Nat. Acad. Sci.*, vol. 111, no. 37, pp. 13290–13294, 2014.
- [4] **C. Xin**, Z. Liu, X. Zuo, M. C. Zhou and B. Jia, “Meal delivery routing problem with a hybrid fleet of riders and autonomous vehicles under dynamic environment,” *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 11642–11655, 2025.
- [5] **D. He**, H. Hong, X. Li, Y. Zhang and M. Wang, “Crowdsourcing incentives for multi-hop urban parcel delivery network,” *IEEE Access*, vol. 7, pp. 26255–26267, 2019.
- [6] **E. Ngharamike**, K. P. Seng, L.-M. Ang and E. Peter, “Ridesharing and crowdsourcing for smart cities: Technologies, paradigms and use cases,” *IEEE Access*, vol. 11, pp. 18038–18081, 2023.
- [7] **H. W. Kuhn**, “The Hungarian method for the assignment problem,” *Naval Res. Logist. Quart.*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [8] **J.-F. Chen**, T.-H. Chang and Y.-H. Chang, “An imitation learning-enhanced iterated matching algorithm for on-demand food delivery,” *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 21803–21817, Nov. 2022.
- [9] **J. J. Q. Yu**, S. Zhang and C. Markos, “Autonomous vehicle intelligent system: Joint ride-sharing and parcel delivery strategy,” *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 17457–17472, Oct. 2022.
- [10] **L. Perron** and V. Furnon, “OR-Tools,” Google. [Online]. Available: <https://developers.google.com/optimization/>
- [11] **M. Furuhashi**, M. Dessouky, F. Ord    ez, M.-E. Brunet, X. Wang and S. Koenig, “Ridesharing: The state-of-the-art and future directions,” *Transp. Res. Part B*, vol. 57, pp. 28–46, 2013.

- [12] **V. Aggarwal**, M. Haliem, G. Mani and B. Bhargava, “A distributed model-free ride-sharing approach for joint matching, pricing, and dispatching using deep reinforcement learning,” *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 12, pp. 7933–7951, Dec. 2021.